

Enhancing Software Testing with Ontology Engineering Approach

1st Suraiya Charoenreh
College of Computing,
Prince of Songkla University
Phuket, Thailand
6030223005@psu.ac.th

2nd Adisak Intana
College of Computing,
Prince of Songkla University
Phuket, Thailand
adisak.i@phuket.psu.ac.th

Abstract—This paper presents a novel hybrid framework, **Software Requirement Ontologies based Test Case Generation (ReqOntoTestGen)** to increase the confidence in the reliability of existing verification and validation (V&V) techniques. This framework integrates the benefits of ontology modelling with the test case generation approaches based on use case-based requirement specifications. ROO (Rabbit to OWL Ontologies Authoring) tool is used in this work to eliminate the ambiguous requirement in natural language by using Controlled Natural Language (CNL). The ontology result from this tool, then, is translated into OWL before this OWL model is mapped into the XML file of data dictionary. Test cases are generated from this XML file by using Combination of Equivalence and Classification Tree Method (CCTM). This testing technique enables the redundant test cases to be eliminated and the coverage of testing to be increased. The contribution of this work has been explored by using the real case study. The result shows how the requirement ontology enhances the testing technique as we expected.

Keywords—test case, requirement ontology, software requirement specification, software testing, black-box testing

I. INTRODUCTION

Software testing is one of the main phases in software engineering that enables the software developer to check and evaluate the functionality of the developed software application performing error-free and meets the users' need [1, 2]. Black box testing is a software testing method in which functionality of the System Under Test (SUT) is tested without knowing the internal code structure and implementation details [1, 2]. This type of testing selects test cases from software requirements and specifications. However, requirement specifications are normally described in the use of informal languages e.g. natural languages [3]. This can cause many defects in the requirement specification including unclear and ambiguous requirement specifications. Consequently, this leads to incorrect test cases to be encountered. Therefore, the very clear and well-defined structure of requirement specifications is needed.

Ontology engineering is a set of activities for building ontologies. The ontology is a formal representation of the entities and relationships in the domain of interest [4]. An ontology can be used as a formal specification of a program in which the domain vocabularies, the essential concepts with their classifications, relationships among the concepts (as well as constraints) and domain axiom related to a particular program

application are defined [4, 5]. Thus, the use of ontology to represent the requirement specification influences the benefit in which the ambiguity, inconsistency or incompleteness in requirement specifications to be coped [3, 5-6]. Consequently, this paper is interested in tackling the problem in software testing that of generating incorrect test cases from the ambiguous, inconsistency or incomplete requirement specification by combining the ontology modelling with the test case generation approaches.

Several research studies were have dealt with applying ontologies to software engineering. In [7], the software engineering ontology was defined in order to use as a communication protocol to exchange the semantic project information for multisite software development. [5] proposed *OntoPersonaURM* to explicit software requirements providing the general concept of user's knowledge and characteristics. In [8], the ontologies in requirements engineering was defined in order to clarify the process and activity within traditional requirements engineering and software engineering in particular. [3] presented pattern by using controlled syntax to writing Thai software requirements specification syntactically correct and understood by software engineers. [9] proposed *OntoTest* to sharing knowledge of activity with defining a common vocabulary for software testing. [10] proposed *ROoST* pattern to manage knowledge in the software testing process. Moreover, they presented testing techniques for test case design in order to apply to generate test case. However, from the above literature survey, we conclude that most of the research studies focused on the application of ontologies in the structure of activities and related procedures components of activity in software engineering process. Based on literature reviews described above, there are few attempts to focus on the representation of the semantic functional requirement in SRS document and apply this into software testing aspects. The closest comparable work is [11] which provides to the combine an inference to software requirements ontology to generate test cases based on software requirements specification. This paper proposed the relationship between testing step and testing procedure including generate test case. However, this paper did not provide coverage in test cases generation.

This paper proposes a **Software Requirement Ontologies based Test Case Generation (ReqOntoTestGen)**, a novel hybrid framework to enhance the current practice of V&V techniques.

It combines the benefits of ontology modelling with the test case generation approaches based on use case based requirement specifications. In this work, ROO (Rabbit to OWL Ontologies Authoring) tool, is used to construct an ontology. This tool is a Controlled Natural Language (CNL) based tool that reduces the complexity of requirements described in the natural language before creating the ontology and translated it into OWL. This results in the unclear and ambiguous requirements to be eliminated. The generated OWL model, then, is mapped into the XML file to create a data dictionary which is used as information to generate test cases. To enable the effectiveness of software testing, CCTM (Combination of Equivalence and Classification Tree Methods) is used for test case generation. This method basically provides the benefit which the redundant test case is removed and the coverage of testing for all possible system scenario is discovered.

II. BACKGROUND

A. Ontology

Ontology is a model of application for defining vocabulary under the scope of interests. There are classifications, the relationships among the concepts including the constraint and domain axiom related within the domain [4, 5]. The ontology can organize the unstructured knowledge to be more formal structured which defines the concept, classes and its inheritance as well as relationship and data property. The ontology enables understanding between humans and computers in order to cooperate effectively together. Fig. 1 shows the example of semantic and relationship of ontology *Man* and *Woman* are *Person* and *Male* and *Female* as *Sex*, while *Male* is *Sex* of *Man* and *Female* is *Sex* of *Woman*.

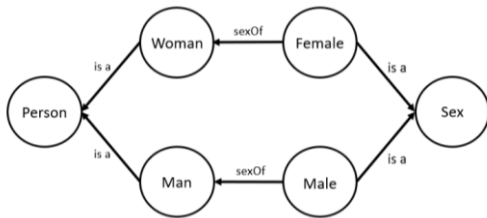


Fig. 1. The Semantic and Relationship of Ontology

B. Rabbit to OWL Ontology authoring (ROO)

ROO^a is a tool for generating ontology, which is a plugin of Protégé that helps domain experts to easily design and build ontology without having a very specific knowledge of ontology design. It can also explain the domain of things that are freely interested in natural sentences with clearly defined syntax of English languages [12]. ROO uses Controlled Natural Language (CNL) to define concepts and relationships which can reduce the complexity of knowledge creation and easy to design ontology. ROO provides specific instructions and easy to understand messages to help users enter the correct CNL structure according to grammatical principles. In natural language processing, ROO avoids using linguistic terminology such as determiner or adjective and entity with ambiguity. This benefits the developer to avoid the ambiguity in natural language. Furthermore, ROO has defined the main structure as a concept, relationship, and

instance to create the ontology. As an example, the pattern is defined in Table I.

TABLE I. ROO STRUCTURE

Syntax of ROO	Description
<concept> is a concept. Ex. <i>Customer</i> is a concept. <i>Order</i> is a concept.	Determining concept or class.
<relation> is a relationship. Ex. <i>checkout</i> is a relationship.	Determining relationships.
Every <concept> <relation> <concept>. Ex. Every <i>Customer checkout Order</i> .	Determining relationship between classes.

^a <http://library.phuket.psu.ac.th/>

Table I illustrates describing the class and relationship definition. From the example, there are 2 classes defined as *Customer* and *Order* and the relationship link is *checkout* with the relation of both classes: Every *Customer checkout Order*. The meaning of this sample is a customer can check out the order list etc.

C. Software Requirement Specification (SRS)

Software Requirement Specification is a description of a software system to be developed with its functional and non-functional requirements. The system analyst collects the requirement from the user by writing in the document in natural language. The details in this document are then used to implement the system in the development process. To enable the mutual understanding, the IEEE [13] standard is used as a guideline to design SRS documents. Inside the document, there are various details according to IEEE standards. One of the important sections of the document is the detail of the requirement which are taken into account to develop the system. A functional requirement is a requirement or what the system should do that is the functionality of the system, for example, *Members can borrow items including Books, DVDs or CDs*. Non-functional requirements are other type of requirements that are not the functionality of the system but is used to judge the operation of the system such as safety, reliability, response time, the ability to connect with other systems.

D. Black-box Testing Techniques

Black-box Testing Techniques is a testing technique that test the functionality of the system. This testing technique performs the test without having the knowledge of the internal mechanism of the program. Software testers only understand the system architecture and its specification without accessing to the source code [14].

E. Combination of Equivalence and Classification Tree Method (CCTM)

CCTM is a combination of two techniques of black-box testing, Equivalence Class Partitioning (ECP) and Classification Tree (CTM) [15]. Before generating test cases, a classification tree is created by CTM as shown in Fig. 2. This starts with creating the system/subsystem as a root. Under the root node called *Class* and *Sub-class* respectively. They define

the functionality and its sub-functionalities of the system along the tree branch until it reaches the parent of the leaf node in the tree called a *terminal classification*. The variables of the system belonging to the specific sub-functionality are defined in the terminal classification. The leaf node of the classification tree is called *terminal class* in which defines the range value of variable defined in its parent node (terminal classification). Then, ECP uses the terminal classification and terminal class to create equivalence partitions that cover both valid and invalid data in generating test cases. CCTM influences the benefit that the coverage of all cases of testing is enabled and the overlap of test cases is reduced. For example, function BMI calculation in which a person's weight is measured based on the range of weight and height with BMI between 25.0-29.0. The data is divided into 3 partitions, one valid partition (25.0-29.0) and two invalid partitions (< 25.0 and > 29.0). Test cases are generated from every terminal class.

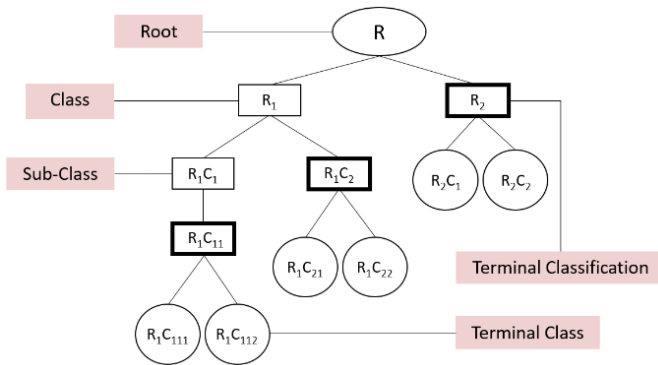


Fig. 2. The Structure of Classification Tree

III. THE PROPOSED METHOD OF REQONTOTESTGEN

Fig. 3 shows the *ReqOntoTestGen* framework. It aims to generate test case based on functional requirement ontology extracted from SRS documents.

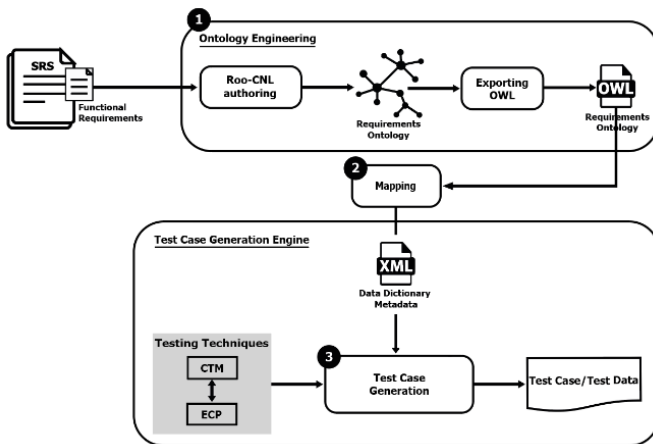


Fig. 3. The *ReqOntoTestGen* Framework

In *Ontology Engineering*, (1) will allow identified functional requirements from the SRS document to be encoded in ontology. The complexity of functional requirements based natural

languages is simplified by using the *ROO-CNL authoring* tool. This results in the complexity of requirements to be transformed into a very simple requirement by using CNL. The *requirement ontology*, then, is generated based on CNL based functional requirement. Next, an OWL model is generated and exported before (2) it is mapped and transformed into the XML of *data dictionary metadata*. In *test case generation*, (3) *CCTM* test case generation technique is used to generate test data.

IV. CASE STUDY FORMULATION

A. Details of Case Studies

To demonstrate how our *ReqOntoTestGen* works, the case study based on the real system is formulated. We chose the library management system of Prince of Songkla University^b to be our effective case study. We conducted 21 requirements and 4 constraints regarding the functional of borrowing and returning items.

TABLE II. FUNCTIONAL REQUIREMENTS OF LIBRARY SYSTEM

Req. ID	Requirements
LIB-FUN-01	Members can borrow items including books, DVDs or CDs.
LIB-FUN-02	Members are classified into admin staff, lecturers, undergraduate and graduate students.
LIB-CON-01	Books, DVDs, CDs must be disjointed.
LIB-CON-02	Admin, staff, lecturers, undergrade and graduate students must be disjointed.
LIB-CON-03	Maximum borrowing items and borrowing periods: 5 books per 7 days for admin staff and undergraduate students, 10 books per 14 days for graduate students, and 15 books per 30 days for lecturers.
LIB-CON-04	Maximum borrowing items and borrowing periods: 3 discs per 7 days for all members.
LIB-FUN-03	Members can return the borrowed item.
LIB-FUN-04	If there is late return, members must pay fine.

Table II shows an example of functional requirements used to demonstrate in this paper. The label containing “*FUN*” and “*CON*” is used in requirement ID (*Req. ID*) to distinguish the functional requirement and constraint respectively. For example, the requirement of operation is labeled as *LIB-FUN-01*, whereas the constraints of this operation which are the maximum number of borrowing item and that of borrowing days length for each member is labeled as *LIB-CON-03* and *LIB-CON-04* respectively. Constraints *LIB-CON-01* and *LIB-CON-02* were added after we analyzed the requirement to create the constraint that all subclasses must be disjointed.

TABLE III. BUSINESS PROCESS OF BOOKS TO LOAN

Categories of Membership	Book		DVD/CD	
	number	days	number	days
Admin Staff	5	7	3	7
Lecturer	15	30		
Undergraduate Student	5	7		
Graduate Student	10	14		

^b <http://library.phuket.psu.ac.th/>

Table III illustrates the business rule describing the constraint of the maximum number of borrowed item and that of

borrowing days. This table also corresponds to constraints *LIB-CON-03* and *LIB-CON-04*.

V. TEST CASE GENERATION IN REQONTOESTGEN

A. Ontology Engineering Process

The *ROO-CNL* authoring tool is used to generate an ontology model from the natural language based functional requirement and its corresponding constraint. The CNL in this tool enables us to simplify the complex requirement by splitting it into a single *Controlled Natural Statement*. For example, in requirement *LIB-FUN-01* “Members can borrow items including books, DVDs or CDs.” is split into two controlled natural statements as “Member is a concept” and “Item is a concept”. After the controlled natural statement is identified in the tool, the ontology and its corresponding OWL model are generated automatically. Table IV shows an example of some mapping of *ROO-CNL* structure implied from functional requirements with its corresponding generated *OLW2 functional syntax*. As can be seen in this table, the concept of *ROO-CNL* is generated into a class in ontology. The *ROO-CNL* authoring tool can create the component of ontology such as a class and its subclass (*SubclassOf*), the disjoint property (*DisjointClasses*), the object property (*ObjectProperty*), the data property (*DataProperty*), existential (*ObjectSomeValuesFrom/DataSomeValuesFrom*), universal (*ObjectAllValuesFrom/DataAllValuesFrom*), and cardinality (*ObjectMinCardinality/ObjectMaxCardinality* and *DataMinCardinality/DataMaxCardinality*).

TABLE IV. ROO STRUCTURE BASED ON LIBRARY REQUIREMENTS

Req.ID	ROO-CNL Structure	OWL2 Functional Syntax
LIB-FUN-01	Member is a concept Item is a concept	Declaration(Class(.Member)) Declaration(Class(.Item))
	Every Book is a kind of Item Every DVD is a kind of Item Every CD is a kind of Item	SubClassOf(.Book .Item) SubClassOf(.CD .Item) SubClassOf(.DVD .Item)
	borrow is a relationship Every Member borrow Item	Declaration(ObjectProperty(.borrow)) ObjectPropertyDomain(.borrow .Member) ObjectPropertyRange(.borrow .Item)
	DataProperty(.hasBorrowDate) DataPropertyDomain(.hasBorrowDate .Member) DataPropertyRange(.hasBorrowDate xsd:dateTime)	Declaration(DataProperty(.hasBorrowDate)) DataPropertyDomain(.hasBorrowDate .Member) DataPropertyRange(.hasBorrowDate xsd:dateTime)
LIB-FUN-02	Every AdminStaff is a kind of Member Every Lecturer is a kind of Member Every UnderGrad is a kind of Member Every Grad is a kind of Member	SubClassOf(.AdminStaff .Member) SubClassOf(.Lecturer .Member) SubClassOf(.UnderGrad .Member) SubClassOf(.Grad .Member)
LIB-CON-03	borrowDaysBookR2 is a Lecturer DataPropertyAssertion is configured data property assertion directly through GUI in the tool.	Declaration(NameIndividual(.borrowDaysBookR2)) ClassAssertion(.Lecturer .borrowDaysBookR2) DataPropertyAssertion(.hasMaxDaysBook .borrowDaysBookR2 30^^xsd:integer)
LIB-FUN-03	return is a relationship Every Member return Item	Declaration(ObjectProperty(.return)) ObjectPropertyDomain(.return .Member) ObjectPropertyRange(.return .Item)
	DataProperty(.hasReturnDate) DataPropertyDomain(.hasReturnDate .Member) DataPropertyRange(.hasReturnDate xsd:dateTime)	Declaration(DataProperty(.hasReturnDate)) DataPropertyDomain(.hasReturnDate .Member) DataPropertyRange(.hasReturnDate xsd:dateTime)
LIB-FUN-04	DataProperty(.hasFine) DataPropertyDomain(.hasFine .Member) ObjectSomeValuesFrom(.hasFine .Member) DataPropertyRange(.hasFine xsd:float)	Declaration(DataProperty(.hasFine)) DataPropertyDomain(.hasFine .Member) ObjectSomeValuesFrom(.hasFine .Member) DataPropertyRange(.hasFine xsd:float)

c. <https://protege.stanford.edu/>

For example, *ObjectProperty* *return* corresponding to *Req. ID LIB-FUN-03* was generated from *ROO-CNL* statement “return is a relationship” before the *ObjectPropertyDomain*

(*Member*) and *ObjectPropertyRange*(*Item*) were defined from *ROO-CNL* statement “Every Member return Item”. *DataProperty* *hasReturnDate* was also defined which has *ObjectPropertyDomain* to be *Member* mapping to *DataPropertyRange* *xsd:dateTime*. Furthermore, as defined in *LIB-CON-03*, *DataPropertyAssertion* for *DataProperty* *hasMaxDaysBook* was specified in order to represent the maximum loan period for *Lecturer* to be 30 days (*30^^xsd:integer*).

The generated OWL model is exported from the *ROO-CNL* authoring tool and validated with Protégé^c tool. Fig. 4 demonstrates the visualization of library ontology after the validation.

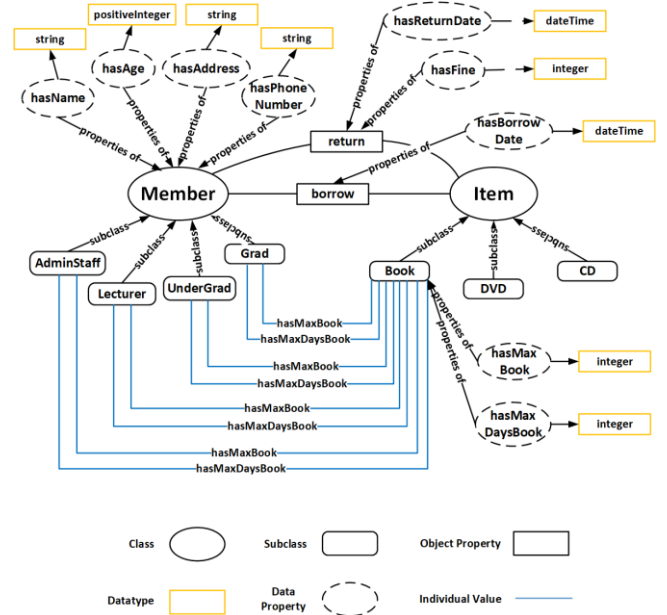


Fig. 4. Example of Ontology Structure of Library System

B. Mapping OWL to XML Process

To enable a common format exchanging in different domain, the generated OWL ontology is transformed into the XML model to create data dictionary. We applied the mapping rule proposed in [16, 17] to generate OWL ontology to XML process. 13 rules are created and used for this mapping step. Table V shows an example of OWL and XML mapping rules. Rule for object property mapping, for example, the domain (defined in *ObjectPropertyDomain*) and range (defined in *ObjectPropertyRange*) of OWL object property is transformed into two elements (e.g. *cdomain* and *crange*) respectively. The *ObjectProperty* *hasclass* in OWL can be mapped into subelement *hasclass* in the XML complex type of element *cdomain*. Furthermore, OWL *ObjectPropertyRange* *crange* is specified into attribute *ref* of subelement *hasclass*.

Similar to the rule for object property mapping, in the rule for data property mapping, the *DataProperty* *dprop* in OWL is mapped into XML subelement *dprop* of element *cname*. *Data Type* defined in OWL *DataPropertyRange* is transformed into attribute *type* of this subelement.

TABLE V. EXAMPLE OF OWL AND XML MAPPING RULES

Rules	OWL2 Functional Syntax	XML Schema
The structure of Classes and Relations		
Class & Subclass Declaration	Declaration(Class(<i>cname</i>)) SubClassOf(<i>scname</i> : <i>cname</i>)	<xs:element name=" <i>cname</i> " type=" <i>scname</i> "/>
Object Property	Declaration(ObjectProperty(<i>hasclass</i>)) ObjectPropertyDomain(<i>hasclass</i> : <i>cdomain</i>) ObjectPropertyRange(<i>hasclass</i> : <i>crange</i>)	<xs:element name=" <i>cdomain</i> " type="xs:complexType"> <xs:sequence> <xs:element name=" <i>hasclass</i> " ref=" <i>crange</i> " /> </xs:sequence> </xs:complexType> <xs:element name=" <i>crange</i> " type="xs:complexType"> <xs:sequence> <xs:complexType> </xs:sequence> </xs:complexType>
Data Property	Declaration(DataProperty(<i>dprop</i>)) DataPropertyDomain(<i>dprop</i> : <i>cname</i>) DataPropertyRange(<i>dprop</i> : <i>sdtype</i>)	<xs:element name=" <i>cname</i> " type="xs:complexType"> <xs:sequence> <xs:element name=" <i>dprop</i> " type="xs:dtype" /> </xs:sequence> </xs:complexType> <xs:element name=" <i>sdtype</i> " type="xs:complexType"> <xs:sequence> <xs:complexType> </xs:sequence> </xs:complexType>
Object Property Restrictions		
Existential	EquivalentClasses(<i>cdomain</i> : <i>ObjectSomeValuesFrom</i> (<i>hasclass</i> : <i>crange</i>))	<xs:element name=" <i>cdomain</i> " type="xs:complexType"> <xs:sequence> <xs:element name=" <i>hasclass</i> " ref=" <i>crange</i> " /> </xs:sequence> </xs:complexType> <xs:element name=" <i>crange</i> " type="xs:complexType"> <xs:sequence> <xs:choice> <xs:complexType> <xs:choice> </xs:choice> </xs:sequence> </xs:complexType>
...	...	...
Data Property Restrictions		
Maximum Cardinality	EquivalentClasses(<i>cname</i> : <i>DataMaxCardinality</i> (<i>max</i> : <i>dprop</i> : <i>sdtype</i>))	<xs:element name=" <i>cname</i> " type="xs:complexType"> <xs:sequence> <xs:element name=" <i>dprop</i> " type="xs:dtype" /> </xs:sequence> </xs:complexType> <xs:element name=" <i>sdtype</i> " type="xs:complexType"> <xs:sequence> <xs:complexType> </xs:sequence> </xs:complexType>

TABLE VI. EXAMPLE OF MAPPING RULES OF LIBRARY SYSTEM

(a) OWL2 Functional Syntax	(b) XML Schema
Declaration(Class(<i>Member</i>)) SubClassOf(<i>Lecturer</i> : <i>Member</i>)	<?xml version="1.0" encoding="UTF-8" ?> <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"> <xs:element name="Member" type="Lecturer" /> <xs:complexType> <xs:sequence> <xs:element name="borrow" ref="Item" /> </xs:sequence> </xs:complexType> <xs:element name="hasMaxBook" type="xs:integer" value="15" /> <xs:element name="hasBorrowDate" type="xs:dateTime" /> <xs:element name="return" ref="Item" /> <xs:element name="hasMaxDaysBook" type="xs:integer" value="30" /> <xs:element name="hasReturnDate" type="xs:dateTime" /> <xs:sequence> <xs:complexType> </xs:sequence> <xs:element> <xs:element name="Item" type="Book" /> <xs:complexType> <xs:sequence> <xs:complexType> </xs:sequence> </xs:element> </xs:sequence> </xs:schema>
Declaration(ObjectProperty(<i>borrow</i>)) ObjectPropertyDomain(<i>borrow</i> : <i>Member</i>) ObjectPropertyRange(<i>borrow</i> : <i>Item</i>)	
Declaration(NameIndividual(<i>borrow</i> : <i>BookR2</i>)) ClassAssertion(<i>Lecturer</i> : <i>borrow</i> : <i>BookR2</i>) DataPropertyAssertion(<i>hasMaxBook</i> : <i>borrow</i> : <i>BookR2</i> : 15 ~xsd:integer)	
Declaration(DataProperty(<i>hasBorrowDate</i>)) DataPropertyDomain(<i>hasBorrowDate</i> : <i>Member</i>) ObjectSomeValuesFrom(<i>borrow</i> : <i>Member</i>) DataPropertyRange(<i>hasBorrowDate</i> : xsd:dateTime)	
Declaration(ObjectProperty(<i>return</i>)) ObjectPropertyDomain(<i>return</i> : <i>Member</i>) ObjectPropertyRange(<i>return</i> : <i>Item</i>)	
Declaration(NameIndividual(<i>borrow</i> : <i>DaysBookR2</i>)) ClassAssertion(<i>Lecturer</i> : <i>borrow</i> : <i>DaysBookR2</i>) DataPropertyAssertion(<i>hasMaxDaysBook</i> : <i>borrow</i> : <i>DaysBookR2</i> : 30 ~xsd:integer)	
Declaration(DataProperty(<i>hasReturnDate</i>)) DataPropertyDomain(<i>hasReturnDate</i> : <i>Member</i>) ObjectSomeValuesFrom(<i>return</i> : <i>Member</i>) DataPropertyRange(<i>hasReturnDate</i> : xsd:dateTime)	

C. Test Case Generation Process

In this step, CCTM technique is used to generate test cases from the XML file of data dictionary achieved from the previous step.

a) Test Case Generation with CTM Technique

Firstly, the classification tree is created by using CTM technique enabled by CCTM. This starts with the name of the

system (*Library System*) is created as a root of the tree. Then, the system functionality is defined as a class of the tree. As we have tackled with the function of fine calculation in this paper, function *fineCalculate* is considered to be a class. Next, in Table VI (b), terminal classifications are created. To accomplish this, the main elements and subelement that contains attribute type (normally are classes and data properties in the ontology) are extracted from the XML file in the data dictionary. This results in *Member*, *Item*, *hasBorrowDate*, *hasReturnDate*, *hasMaxDaysBook* to be created as terminal classification in the classification tree. Furthermore, variables indicate the output of the system including *daysOfLateReturn* and *fine* are also added as terminal classifications. Finally, the range value of each variable defined in terminal classifications is considered together with the value of attribute type in XML elements (e.g. type *Lecturer* in element *staff* and type *xs:integer* with its value 30) to create a terminal class.

CTM technique provides the benefit in which the redundancy of test cases to be eliminated. Based on constraints *LIB-CON-3* and *LIB-CON-4*, there were two trees containing the same terminal classification (*Item*) to be created in order to differently calculate fine for the late return of either books or CDs/DVDs. This enables us to reveal these redundant terminal class and eliminate them by combining these duplicated classifications into the same tree.

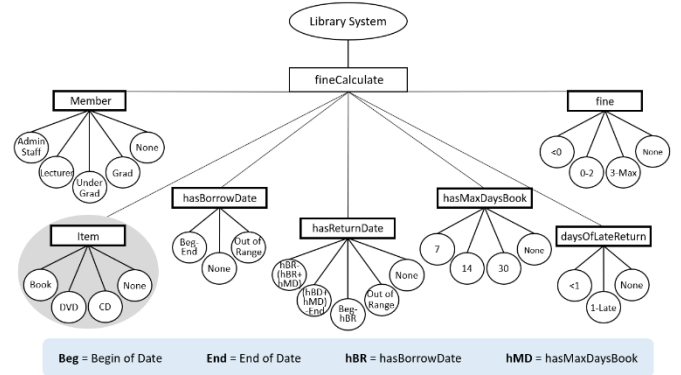
Fig. 5. The Classification Tree of Library System (indicating only *fineCalculate*)

Fig 5 shows the final classification tree after passing the CTM technique. This tree contains five terminal classifications for test input and two terminal classifications for expected output as described above. The terminal class of each terminal classification, defined as a leaf node, is used to be as a partition for generating test cases and test data in ECP. This includes valid and invalid test cases/test data. For example, in terminal classification *Member*, there are four possible values that are *AdminStaff*, *Lecturer*, *UnderGrad* and *Grad* to be valid test cases and *None* (being not a member) to be invalid test cases.

b) Test Case Generation with ECP Technique

The variables and its range values defined as terminal classifications and classes in the classification tree are used to divide partitions and finally generate test cases as shown in Table VII.

TABLE VII. THE PARTITION OF VARIABLES

Partition	Num of Partition	Valid Data				Invalid Data	
Member (input)	5	AdminStaff ⁽¹⁾	Lecturer ⁽²⁾	UnderGrad ⁽³⁾	Grad ⁽⁴⁾	None ⁽⁵⁾	
Item (input)	4	Book ⁽⁶⁾	DVD ⁽⁷⁾	CD ⁽⁸⁾		None ⁽⁹⁾	
hasBorrowDate (input)	3	Begin of Date – End of Date ⁽¹⁰⁾				None ⁽¹¹⁾	Out of Range ⁽¹²⁾
hasReturnDate (input)	5	hasBorrowDate – ⁽¹³⁾ (hasBorrowDate + hasMaxDaysBook)		(hasBorrowDate + ⁽¹⁴⁾ hasMaxDaysBook) – End of Date		Begin of Date – ⁽¹⁵⁾ hasBorrowDate	None ⁽¹⁶⁾ Out of Range ⁽¹⁷⁾
hasMaxDaysBook (Fix rate)	4	7 ⁽¹⁸⁾		14 ⁽¹⁹⁾	30 ⁽²⁰⁾	None ⁽²¹⁾	
daysOfLateReturn (Output)	3	1 – Late Return ⁽²²⁾		None ⁽²³⁾		Smallest Number – 1 ⁽²⁴⁾	
fine (Output)	4	1 x Fine Rate – ⁽²⁵⁾ daysOfLateReturn x Fine Rate		None ⁽²⁶⁾		Smallest number – 0 ⁽²⁷⁾	0 – 1 x Fine Rate ⁽²⁸⁾

There are 28 partitions created by ECP. This includes valid and invalid partitions. Considering the partitions of variable *hasReturnDate*, for example, there are two valid partitions. One for no fine in which members must return the book within the maximum loan period (from *hasBorrowDate* to *hasBorrowDate* + *hasMaxDaysBook* as in partition 13). The other for having fine in which the return date (*hasReturnDate*) is overdue (from *hasBorrowDate* + *hasMaxDaysBook* to End of Date, partition 14). One of the invalid partitions is that it is impossible that the return date is before the borrow date as described in partition 15. Furthermore, not specifying the return date as None and out of range e.g. 31/04/2019 are considered as the invalid partition as in partition 16 and 17 respectively.

TABLE VIII. EXAMPLE OF TEST CASE AND TEST DATA

Test Case	Partition	Member	Item	hasBorrowDate	hasReturnDate	hasMaxDaysBook	daysOfLateReturn	fine	Comments
1	1,6,10,13,18,22,25	AdminStaff	Book	1/7/2019	2/7/2019	7	0	0	Valid (no late return)
2	1,6,10,14,18,22,25	AdminStaff	Book	5/7/2019	15/7/2019	7	3	9	Valid (late return)
...	...	...	...	...	...	...	...	...	...
61	2,6,10,13,20,22,25	Lecturer	Book	1/8/2019	7/8/2019	30	0	0	Valid (no late return)
62	2,6,10,14,20,22,25	Lecturer	Book	1/7/2019	5/8/2019	30	5	15	Valid (late return)
63	2,6,10,15,20,24,27	Lecturer	Book	10/7/2019	9/8/2019	30	1	3	Invalid (return date)
64	2,6,10,16,20,23,26	Lecturer	Book	10/7/2019	32/7/2019	30	None	None	Invalid (none return date)
65	2,6,10,17,20,23,26	Lecturer	Book	10/7/2019	32/7/2019	30	None	None	Invalid (return date out of range)
...	...	...	...	...	...	...	...	...	...
121	3,6,10,13,18,22,25	UnderGrad	Book	5/6/2019	9/6/2019	7	0	0	Valid (no late return)
122	3,6,10,14,18,22,25	UnderGrad	Book	5/6/2019	19/6/2019	7	7	21	Valid (late return)
...	...	...	...	...	...	...	...	...	...
183	4,6,10,13,18,22,25	Grad	CD	1/5/2019	3/5/2019	7	2	0	Valid (no late return)
184	4,6,10,14,18,22,25	Grad	CD	10/5/2019	27/5/2019	7	10	30	Valid (late return)

Table VIII shows the generated test cases and test data created by ECP for testing fine calculation of the library management system. 300 test cases were generated. Test case #61-65 are test cases for testing this module with various book return conditions by lecturers. Test case #61 indicates no fine of this return. Test case #62 is the late book return by the lecturer. The borrow date of this book is on 1/7/2019 but the return date of this book is on 5/8/2019 which has five days of late return. The expected calculated fine is 15 (5×3) Bahts. Test case #63 refers to the invalid condition of return date (9/7/2019) in which it is before the borrow date (10/7/2019). Test case #64-65 are also invalid conditions in which the return date is none and out of range (32/7/2019) respectively.

VI. CONCLUSION AND FUTURE WORK

This paper presents a hybrid framework, *ReqOntoTestGen*, for test case generation by using the ontology representing the semantic of functional requirements in the SRS document. This influences the benefit in which the ambiguity and complexity of

requirements explained in natural language to be eliminated by using CNL in the *ROO-CNL authoring* tool. The OWL ontology is also generated by this tool. To enable the standard of data exchange, this OWL ontology is mapped and transformed into the XML file of data dictionary. Test cases and test data are generated from this XML by CCTM technique. Not only the testing coverage is increased but also the redundancy of test cases is eliminated. After exploring our *ReqOntoTestGen* framework with the formulated case study, the result has shown that the complexity of requirements has been eliminated. It also confirms that the generated test cases from requirement ontology also increase the confidence in reliability of software testing process as this enables the possibility of incorrect test case occurrences to be reduced. We believe that our hybrid framework will strengthen and enhance the current practice of testing.

As the contribution of this framework is demonstrated semi-automatically, the automatic tool generation is required for this work. Therefore, the potential future work is to implement the automatic prototype. This will provide the automated OWL-XML transformation and automated test case generation.

REFERENCES

- [1] R. Patton, *Software Testing*. USA: Pearson Group, 2001.
- [2] P. C. Jorgensen, *Software Testing: A Craftsman's Approach*, 4th ed. Auerbach, USA: CRC Press, 2013.
- [3] K. Thongglin, S. Cardey, and P. Greenfield, "Thai software requirement specification pattern," in *IEEE 12th Int. Conf. Intell. SoMeT.*, Sep. 2013, pp. 179-184.
- [4] C. M. Keet, *An Introduction to Ontology Engineering*. Univ. of Cape Town, 2018.
- [5] W. W. Sim and P. Brouse, "Towards an ontology-based persona-driven requirements and knowledge engineering," *Procedia Comput. Sci.*, vol. 36, pp. 314-321, Nov. 2014.
- [6] K. Siegmund, E. J. Thomas, Y. Zhao, J. Pan, and U. Assmann, "Towards ontology-driven requirements engineering," in *Workshop Semantic Web Enable Software Engineering at 10th ISWC.*, Bonn, Oct. 2011.
- [7] P. Wongthongtham, E. Chang, T. Dillom, and I. Sommerville, "Development of a software engineering ontology for multi-site software development," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 8, pp. 1205-1217, Aug. 2009.
- [8] V. Castañeda, L. Ballejos, M. L. Calusco, and M. R. Galli, "The use of ontologies in requirements engineering," *Global J. Res. Eng.*, vol. 10, no. 6, pp. 2-8, Nov. 2010.
- [9] E. F. Barbosa, E. Y. Nakagawa, and J. C. Maldonado, "Towards the establishment of an ontology of software testing," in *Proc. 18th Int. Conf. SEKE.*, Jan. 2006, pp. 522-525.
- [10] E. F. Souza, R. A. Falbo, and N. L. Vijaykumar, "Using ontology patterns for building a reference software testing ontology," in *IEEE 17th Int. EDOCW.*, Sep. 2013, pp. 21-30.
- [11] V. Tarasov, H. Tan, M. Ismail, A. Adlemo, and M. Johansson, "Application of inference rules to a software requirements ontology to generate software test cases," in *13th Int. Workshop OWLED.*, Nov. 2016, pp. 82-94.
- [12] R. Denaux, *Intuitive Ontology Authoring using Controlled Natural Language*. Univ. of Leeds, 2013.
- [13] *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Standard 830-1998, 1998.
- [14] M. E. Khan, "Different approaches to black box testing technique for finding errors," *Int. J. Softw. Eng. Appl.*, vol. 2, no. 4, p. 31, Oct. 2011.
- [15] B. Ramadoss, P. Prema, and S. R. Balasundaram, "Combined classification tree method for test suite reduction," in *Proc. 2nd ICWET.*, 2011, pp. 27-33.
- [16] O. E. Hajjamy, L. Alaoui, M. Bahaj, "XSD2OWL2: Automatic mapping from xml schema into owl2 ontology," *J. Theor. Appl. Inf. Technol.*, vol. 95, no. 8, pp. 17-30, Apr. 2017.
- [17] N. Yahia, S. A. Mokhtar, and A. Ahmed, "Automatic generation of owl ontology from xml data source," *Int. J. Comput. Sci. Issues*, vol. 9, Jun. 2012.